



T.C.
SAKARYA ÜNİVERSİTESİ

BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
PROGRAMLAMA DİLLERİNİN PRENSİPLERİ ÖDEV RAPORU

NÜFUS SİMÜLASYONU

B241210076- Sinan ULUSİNAN

SAKARYA

Mayıs, 2026

Programlama Dillerinin Prensipleri Dersi

NÜFUS SİMÜLASYONU

Sinan ULUSINAN^{B241210076*-1A}

Özet

Bu ödevde, C programlama dili kullanılarak Nesne Yönelimli Programlama prensiplerinin bir benzetimi gerçekleştirilmiş; dinamik bir nüfus artışı ve şehir bölünmesi projesi geliştirilmiştir. C dilinde Nesne Yönelimli Programlama C++ veya C#'deki hallerine benzemediği, olmadığı için farklı yöntemlerle bu sistematik yapılmıştır. Problem, kullanıcının girdiği başlangıç parametrelerine göre şehir, ilçe, mahalle ve kişi hiyerarşisinin kurulması ve yapıların kalıtım ile birbirini miras almasıdır. Yerlesim adında bir temel yapı tasarlanmış; Şehir, İlçe ve Mahalle birimleri bu yapıdan kalıtım alacak şekilde kurgulanmıştır. Veriler, dinamik bellek yönetimi (malloc, realloc, free) kullanılarak iç içe geçmiş pointer dizileriyle yönetilmiştir. Simülasyonun ihtiyaç duyduğu 10.000'den fazla veri, harici bir Java Faker scripti ile üretilerek 4 farklı .txt dosyasına yazılmış ve C programı tarafından dosya okuma işlemleriyle sisteme entegre edilmiştir.

© 2017 Sakarya Üniversitesi.

Bu rapor benim özgün çalışmamdır. Faydalanmış olduğum kaynakları içerisinde belirttim. Herhangi bir kopya işleminde sorumluluk bana aittir.

Anahtar Kelimeler: C Programlama, OOP Benzetimi, Simülasyon, Java Faker Verileri, Nüfus Modellemesi

1. GELİŞTİRİLEN YAZILIM

Bu ödevde, nesne yönelimli dillerdeki sınıf mantığı, C dilindeki struct ve fonksiyon pointerları kullanılarak inşa edilmiştir.

1.1. Altyapı, Makefile ve Veri Yönetimi: Projenin derlenme aşaması MinGW GCC derleyicisi kullanılarak hazırlanan bir Makefile ile otomatize edilmiştir. Java Faker kütüphanesi bu ödevde doğrudan kullanılamayacağı için, yardımcı bir Java kodu yazılarak 10.000'den fazla rastgele isim, yaş ve yerleşim yeri verisi üretilmiş ve 4 tane .txt formatında kaydedilmiştir. C programı, çalışma anında bu dosyaları fopen komutları ile simülasyon boyunca ihtiyaç duyulan verileri dinamik olarak okumuş ve hafızaya almıştır.

1.2. Algoritma ve Tasarım Kararları

- **Kalıtım Benzetimi:** Ödevin temel mimarisi olan kalıtım için Yerlesim adında bir temel yapı oluşturulmuştur. Şehir, İlçe ve Mahalle yapıları, bu Yerlesim yapısını kendi içlerinde ilk eleman olarak tutarak (Yerlesim* super;) isim ve nüfus metotları gibi ortak özellikleri miras almışlardır.
- **Veri Hiyerarşisi ve Dinamik Bellek:** Java'daki ArrayList yapılarının yerini C dilinde struct olarak almıştır. Yeni bir kişi veya mahalle eklendiğinde kapasite kontrolü yapılmış, doluluk durumunda realloc fonksiyonu ile bellek alanı iki katına çıkarılarak güvenli bir veri yönetimi sağlanmıştır.

* Ödev Sorumlusu. Sinan ULUSINAN, B241210076

Mail Adresi: sinan.ulusinan@org.sakarya.edu.tr

- **Polimorfizm ve Fonksiyon Pointerlar:** Yapıların içine eklenen fonksiyon göstericileri, kurucu fonksiyonlarda asıl metotlara bağlanarak nesne davranışları simüle edilmiştir. Kapsülleme mantığını korumak adına metotların asıl gövdeleri static olarak tanımlanmıştır.
- **Şehirlerin Mitoz Bölünmesi:** Bir şehrin nüfusu 1000 ve üzerine ulaştığında Oyun yapısı devreye girmiş; ilçelerin, mahallelerin veya kişilerin yarısını yeni oluşturulan bir şehre aktarmıştır. Özellikle tek bir birimin (1 ilçe veya 1 mahalle) kaldığı durumlarda, nüfusu yarıya bölerek yeni birime taşıma kuralı hassasiyetle uygulanmıştır.
- **C ve Java arasındaki fark:** İlk ödevdeki .txt notum yine bu ödevde de işime yaradı. Ödevdeki değişiklikler Java faker ile rastgele isim verilmesi kısmı yerine dosyadan isim verilmesi ve Yerlesim yapısıydı. Bu sebeple notumu hiç değiştirmeden tekrar kullanıp aynı sistematikler üzerinden matematiksel işlemleri yaptım. Java ile benzerliği ise yine ID belirlemesini static int sayac = 1; ile yaptım.

```

-Öğünde her turun sonunda şehir nüfusları gösterilmelidir , her satıra 5 şehir gelecek şekilde düzenlemeli. 8 il vardı az önce o yüzden 5 yukarda 3 aşağıda olacak. Ve az önceki şartları gerçekleştirildiğinde oluşacak ve ilk tura başlamadan önce nüfus durumu şu şekilde olur :
[24]-[38]-[77]-[45]-[66]
[34]-[41]-[55]
daha sonrasında ilk tur başlar. Tur sonrasında yapılacaklar :
* Katsayı = nüfus değeri/ilk başmaki toplam (24 = 8)
* bu değer , tüm mahallenin nüfusunun kaç kat olacağını gösteriyor
* mahallelerdeki kişi sayısını bu yeni katsayıyla çarpacağız (24 sayısı için 3 kişiydi , 360=18) oluşan yeni sayı da bütün toplam nüfusu arttıracak katsayımız.
*İmdi ise mahalle sayısı ile bu yeni katsayıyı çarpıp yeni nüfusu bulacağız (18 sayısından dolayı 8 mahalle , 8x144 yeni nüfusu)
-İşer bir sebepten bu hesapların toplamı sıfır olursa, o şehirdeki her mahallede nüfus sadece 1 kişi artar.
-her bir şey mesele değil bir şekilde bu mantıkla devam edecek ve sonraki şehirlerin nüfusları arasında görünecektir.
-İkinci her turun sonunda testlenmelidir. Her turdan sonra devam etmek için tusa bastırarak ilk turun sonuçlarını silirebiliriz. Mesela başlangıç değerleri yazdı , tur 1 gerçekleştiri. Tusa başında başlangıç değerleri yerine tur 1 değerleri yukarda yazılır sonra tur 2 değerleri yazılır. Böyle böyle istenilen tura kadar devam ederiz. Eğer tur kalmadıysa da oyun tona diye .....
* Bir turun sonuna geldiğinde tüm şehirlerin nüfuslarına bakıyoruz. Eğer herhangi bir ilin nüfusu 1000 veya daha fazlaya (yani 4 basamaklıysa):
* Yeni oluşan şehre Java Faker kütüphanesi kullanılarak rastgele yeni bir isim veriliyor. Eski şehrin ismi aynı kalıyor.
* İlçeler bölünmüyor , Eğer bölünecek şehir ilçe sayısı çift ise ilçeler tam ortadan ikiye bölünmüyor (Örn: 8 ilçe varsa 4'ü kalıyor, 4'ü yeni şehre gidiyor).
* Eğer bölünecek şehrin ilçe sayısı tek ise yeni şehre giden ilçe sayısı 1 eşik oluyor. Yeni tek ise 1 artırıp 2 ye bölüp sonra 1 saltatarak gönderiyoruz sonraki ilçeye. Daha kolay bir hesaba da olabilir (Örn: 7 ilçe varsa, 4 ilçe eski şehinde kalıyor, 3 ilçe yeni şehre tasınıyor)
* İlçeler kumullarına bağlı mahalle ve nüfusları da alıyor. Bu sebeple de aslında nüfus 2 ye bölünmüş gibi oluyor. Ama il için bir sayı oluşturmayaçaz. Giristeki sayılar gibi bir sayı olmayacak o yüzden sadece ilçeler bölünecek gibi düşünebiliriz.
* Yeni iller listenin sonuna ekleniyor.
** Mesela 1078 nüfuslu tek ilçeli , oyun başındaki 79 sayısından. 7 ilçe.
1078 / 7 = 154 (ilçe başına nüfus). 7 olduğundan 4 ve 3 ilçe olarak ayrılacak, 4x154 ve 3x154 işleminde eski şehir=616 kalır , yeni şehir=462 olarak oluşturulur.
** Mesela 1488 nüfuslu çift ilçeli, oyun başında 80 sayısından. 8 ilçe.
1488 / 8 = 176 (ilçe başına nüfus). 8 olduğundan 4 4 bölünür. 4x176=704. eski ve yeni şehir aynı nüfusla olur.

-----
sistem hiyerarşisi:
oyun sınıfı : tüm simülasyonu yönetecek
şehir sınıfı: ad , toplam nüfus ve ilçelerin listesi
ilçe sınıfı : ad , nüfus ve ilçeye bağlı mahalle listesi
mahalle sınıfı: ad , nüfus ve mahallede yaşayan kişilerin listesi
kişi sınıfı: adı/soyadı , yas , ID (int türü)
* Faker ile oyun başı her kişiye 0-100 arası yas atacak. her tur bitiminde simülasyondaki herkesin yaşı 1 artacak.
* nüfus artınca tur/ölüm ilimne fakeli tahmin ediliyor bu insanlara yeni ilife ve soy isimler verileceği. Yeni gelen nüfusları de yeni değus gibi düşünüp 0 yasında gibi kabul edip tur sonunda her türlü 1 yasına basması olurlar.
* ID değeri eşsiz olacak , kişi sınıfında static sayac yapıp her yeni kişide ID 1 arttırarak ilerlemek en mantıklısı yeni gelen insanlar için de bu sayac çok kolay hayat kurtarır.

-oyun bitiminde en son ekranda son şehrin nüfus durumları yazacak.
-satır sütun seçimi : şehirleri matris gibi düşün, 8 dan başlayarak satır sütunlardan bir değer isteyebilirsin. satır ve sütun değerine göre istenilen şehir seçilimsi olacak
en acililen seçimleri tüm veriler ekrana dokülecek. format şöyle:
Şehir : ad - nüfus : değer
İlçe : ad - nüfus : değer
mahalle : ad - nüfus : değer
KİŞİLER :
ID-ad-yas
ID-ad-yas
... diğer kişiler
MAHALLE : ad - nüfus : değer
KİŞİLER :
ID-ad-yas
ID-ad-yas
... diğer kişiler
... diğer mahalleleri

```

Şekil 1. TXT formatındaki notlarım

2. ÇIKTILAR

2.1. Matematiksel Çıktılar: Kullanıcıdan alınan başlangıç parametreleri doğrultusunda, turlar ilerledikçe her şehrin nüfusu hesaplanmış ve ekrana 5’li sıralanmış bir matris halinde yansıtılmıştır. C dilinde dosya okuma ve bellek tahsisi işlemleri optimize edildiği için yüksek veri hacminde dahi stabil bir performans gözlemlenmiştir. Malloc sebebiyle/sayesinde Java’ya kıyasla daha hızlı bir ekran çıktısı alıyoruz.

2.2. Detaylı Hiyerarşi Çıktısı: Simülasyon bittiğinde, kullanıcının girdiği satır ve sütun değerine karşılık gelen şehrin detaylı dökümü (Şehir => İlçe => Mahalle => Kişiler [ID- İsim Soyisim- Yaş]) konsola yazdırılmıştır. İşlem bitiminde sistem, ayırdığı tüm dinamik bellek alanlarını yıkıcı fonksiyonlar yardımıyla hiyerarşik bir sırayla temizlemiştir.

3. SONUÇ

Bu çalışma sonucunda; Java gibi yüksek seviyeli dillerde arka planda otomatik Garbage Collector ve Sınıf mekanizmalarının temelinde yatan mantık, C dilinde manuel olarak inşa edilerek tecrübe edilmiştir. Şahsi olarak en büyük katkısı, bellek yönetimi ve pointer kavramlarını somut bir projede kullanmak olmuştur. Özellikle şehir bölünmesi algoritmasını C diline uyarlarken; dizilerden eleman çıkarma, belleği yeniden boyutlandırma kısmını “realloc” ve “dangling pointers” oluşumunu engelleme konularında derinlemesine pratik yapılmıştır. Bellek sızıntılarını önlemek için yıkıcı fonksiyonların en alt birimden (Kişi) en üst birime (Oyun) doğru sırayla temizlenmesi gerektiği kavranmıştır. Bu süreç, bir yazılımın donanımla nasıl iletişim kurduğunu anlamamı sağlamış ve ileride geliştireceğim oyun motorları veya büyük ölçekli sistem projeleri için bana çok sağlam bir temel kazandırmıştır. Unreal ve Unity gibi oyun motorlarında kullanmayı planladığım C++ ve C# için de şükretmeme sebep olmuştur. Java az çok C# a benzediği için çok zorlanmamıştım ama C dili cidden azı dolu bir süreçti. Ödev için test verileri yine sınıf grubunda ilk ödevde kullanılan test verileri ve kendi ödevimden aldığım çıktı verileridir. Her veriyle yapılan sonuçlar ilk ödevdekiyle aynı çıkmış ve testler başarıyla yapılmıştır.

Referanslar

[1] Sakarya Üniversitesi, Programlama Dillerinin Prensipleri Dersi Ödev Yönergesi, 2026.

[2] "Nesne Yönelimli Benzetim" ve "Kalıtım Benzetimi", Ders İçi Yardımcı Video Kaynakları.